Compiling for Distributed Memory: A Historical Survey and Taxonomy from Global Arrays to Mesh-Sharded AI Systems

Sameeran Joshi

***Abstract*—Compilers that target machines with physically distributed memories must decide where data lives, where computation runs, and what communication connects the two. These decisions were first posed as compiler problems in the late 1980s for hypercubes and multicomputers, were standardised in High Performance Fortran in the 1990s, lost ground to hand-written message passing in the 2000s, and have returned in the 2020s as SPMD partitioners and tensor-sharding compilers for accelerator meshes and wafer-scale machines. This article surveys that history. We give an operational definition of a distributed-memory compiler, a reproducible literature-discovery methodology with an openly maintained catalog, and a taxonomy with independent axes for programming abstraction, data decomposition, computation mapping, communication synthesis, compiler/runtime boundary, compiler representation, target, and optimization objective. Using the taxonomy, we trace how mechanisms such as alignment and distribution directives, the owner-computes rule, communication-set generation, redistribution, and inspector–executor scheduling have persisted, disappeared, or reappeared under new names such as device meshes, sharding annotations, partition propagation, collective insertion, and resharding. We close with open problems for heterogeneous hierarchies, dynamic and irregular partitioning, memory-capacity- and topology-aware cost models, and fine-grained spatial architectures. [TODO: Add corpus size and number of systems once the catalog is frozen.]**

***Index Terms*—Distributed-memory compilers, data distribution, owner-computes, communication generation, High Performance Fortran, inspector–executor, polyhedral compilation, SPMD partitioning, tensor sharding, wafer-scale compilation.**

I. Introduction

A machine whose memory is distributed forces a compiler to answer three questions that a shared-memory compiler can defer to hardware: where each datum is stored, which place executes each operation, and what messages must flow so that operands are present when they are needed. The first compilers to pose these as compiler problems appeared in 1988 for hypercube and multicomputer targets [1]–[3]. Within five years the answers had been standardised in the directive language High Performance Fortran (HPF) [4], and the message-passing interface MPI had been standardised as the portable alternative in which the programmer answers all three questions by hand [5]. We observe that the same three questions are now being answered again, by SPMD partitioners such as GSPMD [6] and PartIR [7], by distributed tensor-algebra compilers such as DISTAL [8], and by compilers for wafer-scale and spatial accelerators [9], often with terminology that does not acknowledge the earlier work.

Manuscript draft generated from the open catalog at https://github.com/Sameeranjoshi/dist_compilers. This is a working draft; see the repository ROADMAP for status.

A. Why the problem keeps reappearing

Every time a new class of machine with distinct memories becomes important—hypercubes, clusters, multi-GPU nodes, accelerator pods, wafers—the trade-off between programmer control and compiler automation is re-opened, and the compiler community rebuilds a stack of ownership representation, computation mapping, and communication synthesis for that class. The repository that accompanies this survey exists to make that rebuilding less amnesiac: it records, with evidence, which mechanism solved which problem, under what assumptions, and with what limitation.

B. Scope

We define a *distributed-memory compiler* as a compiler, compiler framework, programming system, or compiler-integrated runtime that transforms a program for execution across multiple computational places whose memories are physically or logically distinct at the relevant compilation level (Section II gives the inclusion criteria). Multiple GPUs are a distributed-memory target when the compiler must partition data or computation and generate communication between device memories; wafer-scale and spatial architectures count when individual processing elements have distinct local memories and data movement must be compiled explicitly. Distributed databases, distributed operating systems, cloud schedulers, hardware-only papers, and shared-memory-only compilers are out of scope.

C. Central question and research questions

The central question is: *how have compilers for machines with physically distributed memories represented, decomposed, mapped, communicated, optimized, and executed data and computation?* We refine it into ten research questions (RQ1–RQ10) on programming abstraction, data decomposition and ownership, computation decomposition, communication generation, the compiler/runtime boundary, compiler representation, optimization objectives, evaluation, historical continuity, and open problems. Section V maps each to an axis of the taxonomy.

D. Contributions

1) An operational definition and inclusion criteria for distributed-memory compilers that span 1950–2026 while keeping pre-1980 work explicitly labelled as *foundations* rather than as distributed-memory compilation (Section III).
2) A reproducible survey methodology with an open, schema-validated catalog in which every entry carries evidence pointers and a screening status (Section II).
3) A common architecture for distributed-memory compilers (Section IV) and a multi-axis taxonomy (Section V), both derived from the catalog.
4) A historical narrative organised by era (Section VI) and by mechanism (Sections VII–XI), each ending with the limitation that motivated the next step.
5) A cross-era synthesis identifying ideas that persisted, failed, or were rediscovered under new names (Section XII), and a list of open problems (Section XIII).

[TODO: We do not claim that this is the first comprehensive survey of the area; an explicit search for prior surveys is logged in the repository and the claim will be revisited once it is complete. Known prior overviews include [10]–[12].]

II. Survey Methodology

A. Scope and inclusion criteria

A system is included when it performs or represents at least one of: data distribution or ownership; computation partitioning or placement; communication generation; collective insertion; halo or ghost-region construction; global-to-local address conversion; import/export schedule generation; redistribution or resharding; hierarchical parallelism across nodes or devices; distributed execution-plan generation; or distributed-memory performance modelling used by a compiler. We classify included work as *central* (automatic distributed-memory parallelization; global-array and data-parallel compilers; Fortran D, HPF and relatives; automatic message-passing generation; distributed dense and sparse tensor compilers; SPMD partitioners; distributed GPU/accelerator compilers; spatial and wafer-scale compilers) or *bridge* (languages whose compilers implement distributed memory; compiler-integrated runtimes such as inspector–executor systems; PGAS compilers; task/region systems where placement is central; distributed-training systems with substantial plan generation; performance models that drive compiler decisions). Excluded are distributed databases and operating systems, generic job scheduling, routing protocols, hardware-only papers, application scaling studies, shared-memory-only compilers, single-GPU kernel compilers, and pure runtimes without a compiler component. A small number of architecture papers are retained in the catalog solely to date target classes; they are not classified on the compiler axes.

B. Sources and queries

Bibliographic metadata were taken, in order of preference, from the DOI record (via Crossref), arXiv, official author or institution copies, official project documentation, and trusted bibliographic indexes. No paywall was bypassed; where full text was not available the catalog marks the entry `paywalled` or `abstract-only` and no mechanism is inferred from the title. The initial query set (25 phrases such as “data distribution compiler”, “communication set generation”, “inspector executor distributed memory”, “SPMD partitioner compiler”, “wafer scale compiler”) is listed in the repository together with a discovery log recording each query, date, hits, and screening decision. [TODO: Report per-database hit counts once the full query sweep over ACM DL and IEEE Xplore is logged.]

C. Snowballing

From every anchor paper we snowball backward (references), forward (citing works, prioritising those that extend, critique, evaluate, or revive the anchor), laterally (same system, project, or authors), across surveys (works cited by several independent surveys are prioritised), and by *gap*: for every empty cell of the taxonomy coverage matrix we search for that combination specifically.

D. Screening and evidence levels

Each catalog entry carries one of five statuses: *discovered* (metadata only), *screened* (abstract and introduction checked), *core* (clearly relevant), *read* (enough read to summarise accurately), and *verified* (metadata, categorisation, and summary reviewed against the primary source). Only *read* and *verified* entries support mechanism-level claims in this article. A paper is called a survey only if it is a survey or an invited overview; for example, Lee’s “Techniques for compiling programs on distributed memory multicomputers” [13] is catalogued as a technique paper.

E. Reading strategy and paper cards

For most papers we read the abstract, introduction, system overview, related work, and conclusion; we read the full paper when it introduced a major system, defines a taxonomy category, supports a key historical claim, has experiments that we compare, or supplies a mechanism central to the synthesis. Every core paper receives a paper card in the repository listing problem, input abstraction, target, decomposition, mapping, communication, compiler/runtime split, evaluation, limitation, lineage, and the exact sections used.

F. Evidence labelling

Statements in this article are of three kinds: source-supported facts, which carry a citation; survey taxonomy introduced by us, flagged by “we categorize” or “we group”; and inferences, flagged by “appears”, “we observe”, or “we hypothesise” and listed with their current evidence in the repository’s historical-claims file. Conflicts between sources are recorded rather than resolved silently.

G. Limitations

Full text of many 1988–2000 papers is paywalled; their summaries rely on open author copies where available. Citation counts are not used as a relevance criterion. Dates of era boundaries are provisional and are justified by the dated events listed in the catalog.

III. Foundations (c. 1950–1979)

This section exists to separate what the distributed-memory compilers of the late 1980s *inherited* from what they *invented*. Nothing in this section is a distributed-memory compiler; the machines of the period had a single address space or lock-step SIMD control, and "data placement" was not a compiler concern.

A. Automatic programming and compiler construction

The FORTRAN I compiler of 1957 established that a translator could produce object code competitive with hand coding for numerical loops [14]. Every subsequent attempt to hide a machine's structure from the scientific programmer, including HPF and today's sharding compilers, re-states that bargain: accept a higher-level program in exchange for compiler-chosen low-level decisions.

B. Dependence analysis and loop transformation

Lamport's hyperplane method showed that the iterations of a nested DO loop could be re-ordered and executed concurrently when the dependence structure permits [15], and Kuck and colleagues framed dependence graphs as the central representation for compiler optimisation and parallelisation [16]. Allen and Kennedy's vectoriser applied these ideas to translate FORTRAN into vector form [17]. We treat dependence analysis as the representation foundation (axis K) that Era 2 compilers reused unchanged; what they added was the ownership function.

C. Machine abstractions

The ILLIAC IV array processor [18] and the CRAY-1 vector processor [19] defined the two regular-parallel machine classes that early parallelising compilers targeted. Dennis's dataflow architecture [20] and Kung's systolic arrays [21] introduced explicitly local communication between processing elements with their own state. Systolic and dataflow concepts appear to be the closest ancestors of today's spatial and wafer-scale targets, in which the compiler again schedules neighbour-to-neighbour data movement on a fabric. [TODO: Verify from primary sources which vectorising compilers were shipped for CDC and Cray systems and whether any addressed memory banks as distributed; cite only after reading.]

D. Synthesis

What the foundations supplied: a trusted compiler pipeline, dependence-based legality for loop reordering, and machine models with regular parallel structure. What was missing: any representation of *where* data resides. The first multicomputers with physically private memories, such as the Cosmic Cube [22], made that omission the central compiler problem of Era 1–2.

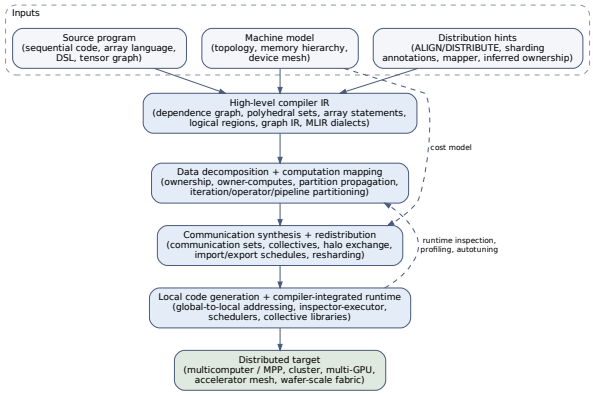


Fig. 1. The stages that recur across distributed-memory compilers from SUPERB to Shardy. The pipeline is linear in most 1990s systems; the dashed feedback edges (runtime inspection, profiling, autotuning, and cost models) are where the compiler/runtime boundary (axis F) is drawn differently by different systems. Figure original to this survey; stages are abstracted from the catalog entries cited in the text.

IV. A Common Compiler Architecture

We abstract the systems in the catalog into the pipeline of Fig. 1. Its inputs are a source program (sequential code, an array language, a DSL, or an operator graph), a machine model (processor arrangement, device mesh, memory hierarchy, topology), and distribution information that is either supplied (ALIGN/DISTRIBUTE directives [4], sharding annotations [6], a mapper interface [23]) or inferred.

a) High-level IR: Era-2 compilers worked on dependence graphs over Fortran loops [24]; polyhedral systems represent iteration and data spaces as integer sets [25]; ML systems use operator graphs [6]; and recent work uses MLIR dialects that carry sharding or distribution attributes [7], [26].

b) Data decomposition and computation mapping: The owner-computes rule, which assigns a statement to the place that owns its left-hand side, appears in the Kali and Fortran D compilers [24], [27]; GSPMD's partition propagation infers shardings of unannotated tensors from annotated ones [6]. Both are instances of deriving a computation mapping from a data mapping; Section VII examines the alternatives.

c) Communication synthesis and redistribution: Compilers derive communication either in closed form from the distribution functions (communication sets and local address generation [28]–[30]), by recognising collective patterns, or at runtime by inspecting indices [31]. Redistribution between phases is a separate stage in HPF compilers [32] and reappears as resharding in SPMD partitioners.

d) Local code generation and compiler-integrated runtime: Era-2/3 compilers emitted SPMD Fortran with calls to message-passing libraries; PARTI/CHAOS supplied runtime schedules for irregular accesses [33]; Legion executes a mapped task graph on a runtime that manages region coherence [23]; XLA-based systems lower to per-device programs with collective calls [6]. We use the amount of work delegated to the runtime as axis F of the taxonomy.

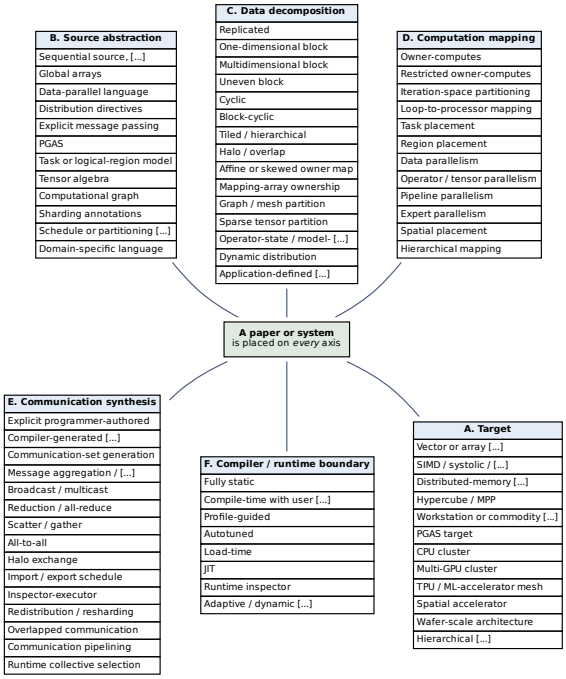


Fig. 2. The taxonomy is a set of orthogonal axes, not a tree: each paper or system takes one or more values on every axis. Only the six axes used most in the text are shown; the catalog carries eleven. Generated from the machine-readable taxonomy so that the figure cannot drift from the catalog.

e) Synthesis: The pipeline has been stable since the early 1990s; what has changed across eras is (i) the input abstraction, (ii) the representation on which decomposition is decided, and (iii) which stages are static versus dynamic. These three differences are exactly axes B, K, and F of the taxonomy.

V. Taxonomy

We classify each paper or system along eleven independent axes (Fig. 2): (A) target and memory model; (B) programming abstraction (RQ1); (C) data decomposition (RQ2); (D) computation mapping (RQ3); (E) communication synthesis (RQ4); (F) compilation time, i.e. the compiler/runtime boundary (RQ5); (G) compiler architecture; (H) optimization objective (RQ7); (I) maturity and artifact status; (J) paper type; and (K) compiler representation (RQ6). The value lists are maintained in the repository and enforced by a validator.

A. Why chronology alone is insufficient

HPF (1993) and GSPMD (2021) take the same values on axis C (block distribution over a logical processor grid) and on axis E (communication inserted by propagating distributions through the program), yet they are three eras apart; inspector–executor runtimes (1990) and distributed sparse-tensor compilers (2022–2025) share the *runtime inspector* value on axis F. A chronological list hides these recurrences, which are among the survey's main findings (Section XII). Chronology is therefore one dimension (Section VI), used to explain *why* a value was chosen at a given time, while the axes explain *what* was chosen.

B. Provenance of category names

Where a standard term exists we use it: *owner-computes* from the Kali/Fortran D literature, *inspector–executor* from PARTI, *BLOCK*, *CYCLIC*, and *block-cyclic* from HPF, *partition propagation* from GSPMD, *inter-* and *intra-operator parallelism* from Alpa [34]. The grouping of these into axes, and the decision to place ML sharding compilers and wafer-scale compilers on the same axes as 1990s HPF compilers, is ours.

C. Tagging discipline

A tag is attached to a catalog entry only when an evidence pointer names the section of the primary source supporting it. Entries whose full text has not been consulted are tagged only on the axes their abstract clearly supports. Lineage fields record only relationships that a paper states itself or that a cited survey states; inferred lineages are kept separately. Table I summarises the eras, Table II the core systems, and Tables III–V the decomposition support, compiler architecture, and evaluation methodology of each system; all are generated from the catalog.

VI. Historical Evolution

Fig. 3 and Table I organise the catalog into seven provisional eras. Each subsection below ends with what changed, what remained, and the limitation that motivated the next era.

A. Era 1: emergence (c. 1980–1987)

Message-passing multicomputers such as the Cosmic Cube [22] and the Connection Machine's data-parallel model [67] made private memories and explicit communication the programmer's problem. [TODO: Add verified early source-to-source and placement work from this period after backward snowballing from [1] and [2].]

B. Era 2: global arrays and data decomposition (c. 1988–1992)

Callahan and Kennedy proposed compiling sequential Fortran with distribution annotations for distributed-memory multiprocessors [1]; SUPERB offered semi-automatic MIMD/SIMD parallelisation with overlap regions [2]; Chen, Choo, and Li compiled parallel programs by optimising a performance model [3]. Kali introduced compiled global-name-space parallel loops [27], Fortran D introduced decompositions, alignment, and distribution with compiler optimisations such as message vectorisation [24], [46], and Vienna Fortran extended Fortran with distribution specifications [48]. Automatic alignment [35] and compile-time distribution selection [68] addressed the choice of distribution, and PARTI introduced runtime inspection for irregular loops [31]. *Synthesis.* What changed: data ownership became a compiler-visible object. What remained: dependence analysis from the foundations. Limitation: regular affine codes only, and no agreed language, which motivated HPF.

TABLE I
ERA SUMMARY. DATES ARE PROVISIONAL (SEE SEC. II); REPRESENTATIVE SYSTEMS ARE THE CATALOG ENTRIES WITH THE HIGHEST READING PRIORITY IN EACH ERA.

Era	Machine driver	Programming abstraction	Decomposition representation	Generated communication	Representative systems	Limitation
0: Foundations (1950–1979)	Stored-program machines, FORTRAN, pipelined vector processors, lock-step array processors, dataflow and systolic proposals	Sequential FORTRAN; vector statements; early parallel-language proposals	None for distribution; dependence graphs and loop-level analysis for vectorization	None (shared or centralised memory)	–	No notion of data placement as a compiler problem; machines had a single address space or SIMD broadcast control
1: Emergence of distributed-memory compilation (1980–1987)	Multicomputers and hypercubes (Cosmic Cube, iPSC, nCUBE), systolic arrays, Connection Machine	Explicit message passing; early global-array proposals; SIMD data-parallel languages	Programmer-managed; first compiler proposals for placement	Hand-written sends/receives	–	Programming by hand was error-prone; portability across machines was poor
2: Global arrays and data decomposition (1988–1992)	Commercial hypercubes and MPPs (iPSC/2, iPSC/860, nCUBE-2, CM-2)	Sequential or global-array programs with distribution annotations (SUPERB, Kali, Vienna Fortran, Fortran D)	BLOCK/CYCLIC distributions, alignment to templates/decompositions, overlap regions	Compiler-generated point-to-point messages; message vectorization; first inspector-executor runtimes	[1], [24], [27], [35]	Regular, affine programs only; automatic distribution selection intractable in general; communication costs poorly modelled
3: HPF, MPI, and compiler maturity (1993–2000)	MPPs (CM-5, Paragon, SP2, T3D/T3E) and early clusters	HPF directives; MPI library calls; irregular extensions	HPF ALIGN/DISTRIBUTE with BLOCK, CYCLIC(k), templates and processor arrangements; INDIRECT/mapping arrays in HPF-2 and research systems	Communication sets, local address generation, redistribution, collectives; PARTI/CHAOS schedules for irregular code	[4], [10], [11], [36]	HPF compilers were slow to mature and performance was unpredictable; explicit MPI became the dominant practice
4: MPI dominance and alternative programming models (2000–2010)	Commodity clusters, early multicore nodes, Cell/BE and explicitly managed memories	MPI(+OpenMP); PGAS (UPC, Co-array Fortran, Titanium); HPCS languages (Chapel, X10); hierarchical memory models (Sequoia)	Language-level distributions and locales/places; hierarchical task trees	One-sided communication, compiler-scheduled bulk transfers for memory hierarchies	[37]–[40]	Adoption of new languages remained limited; automatic distribution rarely used in production
5: Compiler/runtime co-design and accelerators (2010–2019)	GPU clusters, heterogeneous nodes, early TPU pods	Affine loop nests; task/region programs (Legion, Regent); tensor DSLs; ML graph frameworks	Polyhedral partitions; logical regions and partitions; named-dimension layouts (Mesh-TensorFlow)	Polyhedral communication-set generation with MPI; dynamic region coherence; collectives in ML frameworks	[23], [25], [41], [42]	Each system covered one domain; no shared ownership IR; ML partitioning still largely manual
6: Mesh sharding, ML systems, and spatial accelerators (2020–2026)	TPU/GPU pods, AI accelerators, wafer-scale engines, spatial dataflow chips	Sharding annotations on device meshes; tensor algebra with distribution formats; MLIR dialects; stencil DSLs	Device meshes with per-dimension sharding; distributed tensor formats; MLIR shard/mesh attributes	Collective insertion by propagation; resharding; automated search over parallelism plans	[6]–[8], [34]	Dynamic/irregular workloads, heterogeneous hierarchies, and memory-capacity constraints remain weakly handled; fragmentation across frameworks

C. Era 3: HPF, MPI, and compiler maturity (c. 1993–2000)

HPF standardised ALIGN, DISTRIBUTE, TEMPLATE, and PROCESSORS [4]; Fortran 90D/HPF [51] and PARADIGM [36], [50] were research compilers, and vendors shipped production HPF compilers [53], [69]. Communication-set and local-address generation for block-cyclic distributions was solved in closed form [29], [30]; dynamic partitioning and redistribution were optimised [32], [70]; CHAOS extended inspector–executor to adaptive irregular programs [33], [71]. MPI, with MPICH as its portable implementation [5], standardised the explicit alternative. Benkner and Zima's overview summarises the HPF compilation techniques of the decade [11], and Kennedy, Koelbel, and Zima's retrospective analyses why HPF did not achieve lasting adoption [72]. *Synthesis.* What changed: a standard abstraction and mature closed-form communication generation. What remained: owner-computes and block/cyclic distributions. Limitation: compiler performance was unpredictable and irregular codes were poorly served, so practitioners chose MPI.

D. Era 4: MPI dominance and alternative models (c. 2000–2010)

PGAS languages (Co-array Fortran [37], UPC, Titanium) and the HPCS languages X10 [54] and Chapel [55] kept locality in the language but gave the programmer explicit places or locales. Sequoia compiled hierarchical task trees onto explicitly managed memory hierarchies [56], and polyhedral tools such as Pluto [57] and explicitly-managed-memory mapping [73] supplied the representation used by the next era. *Synthesis.* What changed: distribution moved from directives into language types and hierarchies. What remained: the owner/locale concept. Limitation: adoption stayed limited and automatic distribution was rarely used in production. [TODO: Add UPC, Titanium, and tiled-code-generation entries once verified.]

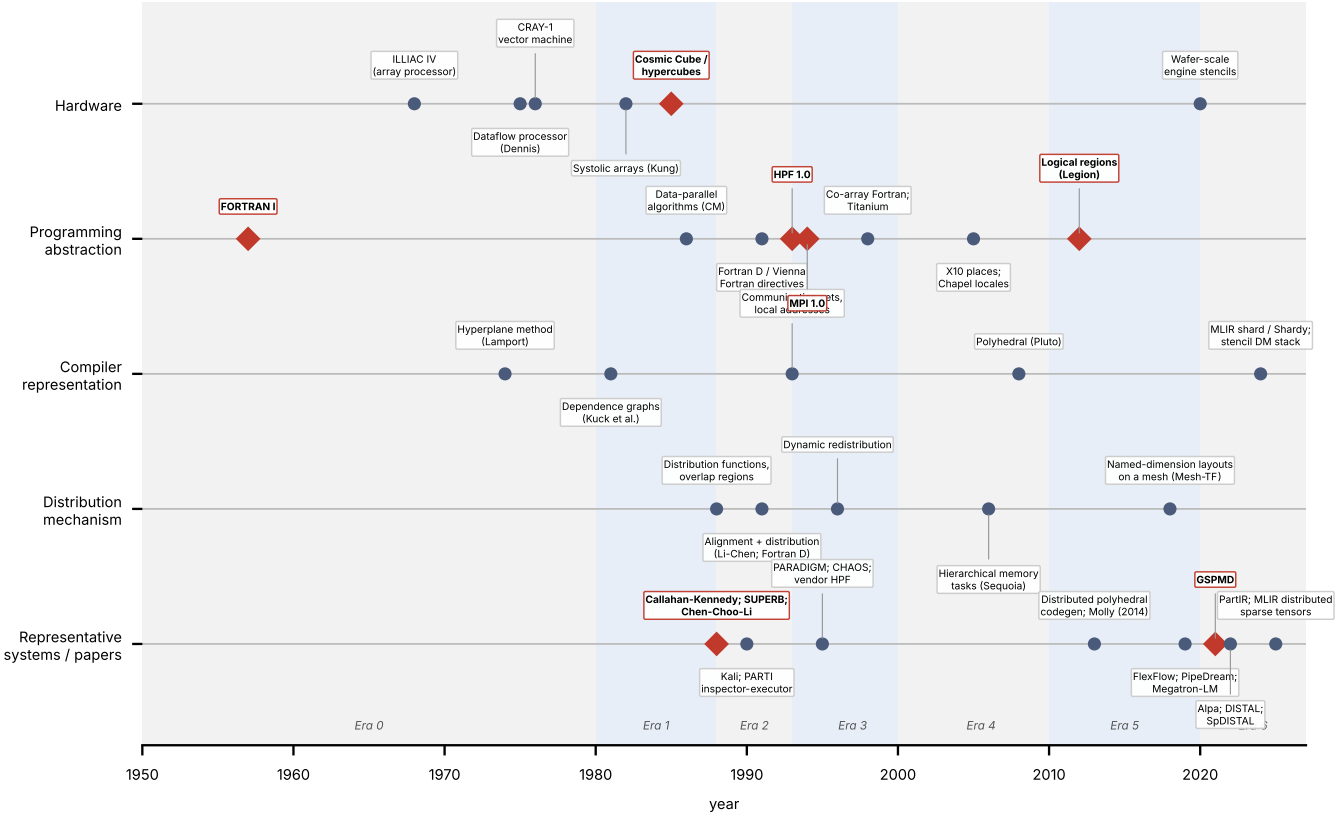


Fig. 3. Timeline 1950–2026 with lanes for hardware, programming abstraction, compiler representation, distribution mechanism, and representative systems. Diamonds mark the transitions that open a new era. Each event is tied to a catalog entry; the era boundaries are provisional (Section II).

E. Era 5: compiler/runtime co-design and accelerators (c. 2010–2019)

Bondhugula generated MPI communication for affine loop nests from polyhedral dependences [25], Molly did so inside LLVM [41], and Dathathri et al. extended this to heterogeneous and dynamically scheduled targets [74]. Legion [23] and Regent [58] made logical regions and mappers the interface between compiler and runtime. Mesh-TensorFlow introduced named-dimension layouts on a device mesh [59]; FlexFlow searched parallelisation strategies automatically [60]; GPipe, PipeDream, and Megatron-LM established pipeline and tensor parallelism in training systems [75]–[77]. *Synthesis.* What changed: accelerators and ML workloads became the driver; representations diversified (polyhedral, regions, graphs). Limitation: each system covered one domain and ML partitioning remained largely manual.

F. Era 6: mesh sharding, ML systems, spatial accelerators (c. 2020–2026)

GSPMD generalised sharding annotations and propagation [6]; Alpa automated inter- and intra-operator parallelism [34]; DISTAL and SpDISTAL compiled dense and sparse tensor algebra with explicit distribution formats [8], [63]; PartIR composed partitioning tactics on an MLIR-based IR [7]; a shared MLIR stack delivered distributed-memory parallelism to stencil DSLs [26]; and wafer-scale stencil codes were compiled onto the Cerebras engine [9]. *Synthesis.* [TODO: Complete after paper cards for Shardy, MLIR shard, PartTensor, and the wafer-scale stencil pipeline are written.]

VII. Data Decomposition and Computation Mapping

A. Representing ownership (RQ2)

We group decompositions into rectilinear, structured non-rectilinear, and irregular (Fig. 4). Fortran D and HPF expressed ownership as an alignment of arrays to a template followed by a distribution of the template over a processor arrangement, using BLOCK, CYCLIC, and CYCLIC(k) [4], [24]. Li and Chen treated alignment as an optimisation problem on a component-affinity graph [35], and Ramanujam and Sadayappan derived distributions from loop dependences at compile time [68]. Irregular ownership was expressed through mapping arrays in PARTI/CHAOS and user-specified irregular distributions [71]. Modern systems express ownership as per-dimension sharding over a device mesh [6] or as tensor distribution formats [8].

B. Assigning computation (RQ3)

Owner-computes assigns each assignment to the owner of its left-hand side [24], [27]; PARADIGM and later polyhedral work partition iteration spaces instead [25], [36]; Legion places tasks through a mapper [23]; ML systems choose among data, operator, and pipeline parallelism [34].

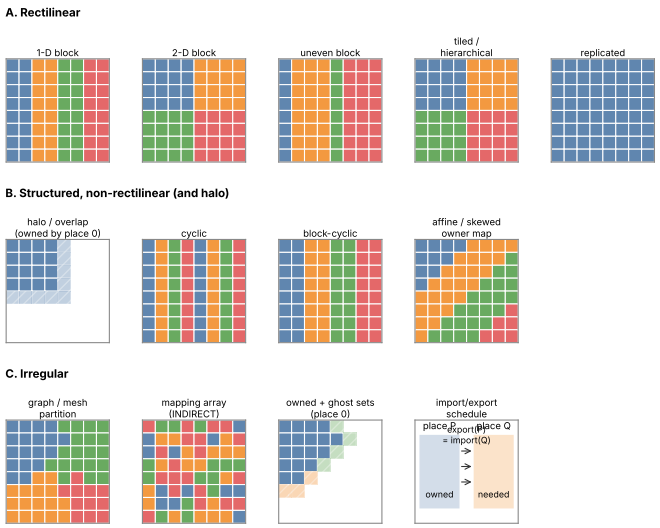


Fig. 4. Data-decomposition taxonomy (axis C) drawn as example grids coloured by owning place. Rectilinear forms (A) are expressible as closed-form owner functions; structured non-rectilinear forms (B) still are, but need block-cyclic address arithmetic; irregular forms (C) require a mapping array or graph partition and, consequently, runtime-built communication schedules. Hatched cells are ghost copies.

C. Synthesis

[TODO: Compare the expressiveness of HPF's distribution language with GSPMD's sharding specs and DISTAL's formats, with evidence from the respective specifications.]

VIII. Communication Generation and Redistribution

Koelbel derived regular communication patterns at compile time for Kali loops [28]; Chatterjee et al. gave closed-form local address and communication-set generation for block-cyclic distributions [29]; Stichnoth, O'Hallaron, and Gross generated communication for array statements and evaluated it in a production compiler [30]. Fortran D's message vectorisation hoisted element messages out of loops [46]. Redistribution between distributions was optimised by Ramaswamy, Simons, and Banerjee [32] and dynamic partitioning by Palermo, Hodges, and Banerjee [70]. Polyhedral compilers compute communication sets as integer sets and emit MPI calls [25], [41]. SPMD partitioners insert collectives where propagated shardings disagree [6], [7].

Synthesis. Communication-set generation and collective insertion solve the same problem on different representations: the former on affine index expressions, the latter on operator semantics. We hypothesise that the loss of generality in the latter (only operator-level patterns) is compensated by much simpler cost modelling. [TODO: Support with evidence from the PartIR and GSPMD evaluations.]

IX. Runtime Analysis and Irregular Computation

Saltz and colleagues introduced the inspector–executor scheme in which an inspector pass examines index arrays to build a communication schedule reused by an executor [31]; the PARTI and CHAOS libraries supplied import/export schedules, translation tables, and schedule reuse that compilers could target [33], [71]. Legion moves the entire decision of data movement to a runtime that reasons about logical regions and privileges [23]. SpDISTAL compiles distributed sparse tensor computations with distribution formats and runtime communication planning [63].

Synthesis. Axis F separates systems more sharply than any other: fully static closed-form systems (HPF, polyhedral) are fast and predictable but limited to affine codes, while runtime-inspected systems (PARTI, Legion, SpDISTAL) handle irregularity at the cost of inspector overhead that must be amortised. [TODO: Quantify inspector overhead from the CHAOS and SpDISTAL evaluations.]

X. Performance Modeling and Optimization Objectives

Chen, Choo, and Li compiled by optimising an explicit performance model [3]; Ramanujam and Sadayappan and Gupta and Banerjee chose distributions from compile-time cost estimates [49], [68]; PARADIGM used a cost model for distribution selection [36]. Alpa formulates intra-operator parallelism as an integer linear program and inter-operator parallelism as dynamic programming over a profiled cost model [34]; FlexFlow uses a simulator-guided search [60].

Synthesis. Axis H records objectives: communication volume and message count dominate Eras 2–3, memory capacity and overlap dominate Era 6. [TODO: Tabulate the objective of each core system with evidence pointers.]

XI. Multi-GPU, ML, Spatial, and AI-Accelerator Compilers

Mesh-TensorFlow [59], GShard [61], and GSPMD [6] established the device mesh and per-tensor sharding specification as the programming abstraction of accelerator pods; PartIR [7] separates partitioning tactics from the program and composes them on an MLIR-based IR. Alpa [34] and FlexFlow [60] search the space of parallelisation strategies automatically, while GPipe, PipeDream, Megatron-LM, and ZeRO [75]–[78] are training systems whose plan generation is largely manual and which we therefore classify as bridge work. DISTAL and SpDISTAL [8], [63] bring distribution formats to tensor algebra; Kong et al. generate distributed-memory mappings for tensor computations automatically [79]; a shared MLIR stack gives stencil DSLs distributed-memory parallelism [26]. On wafer-scale hardware, stencil codes have been mapped onto the Cerebras engine with explicit fabric communication [9], [62].

[TODO: Add Shardy, the MLIR shard dialect, PartTensor, MLIR-AIR, and verified accelerator-vendor compiler publications once their metadata and primary documentation are confirmed.]

XII. Cross-Era Synthesis

A. Ideas that persisted

Logical processor grids, block distributions, owner-derived computation mapping, and the separation of a global index

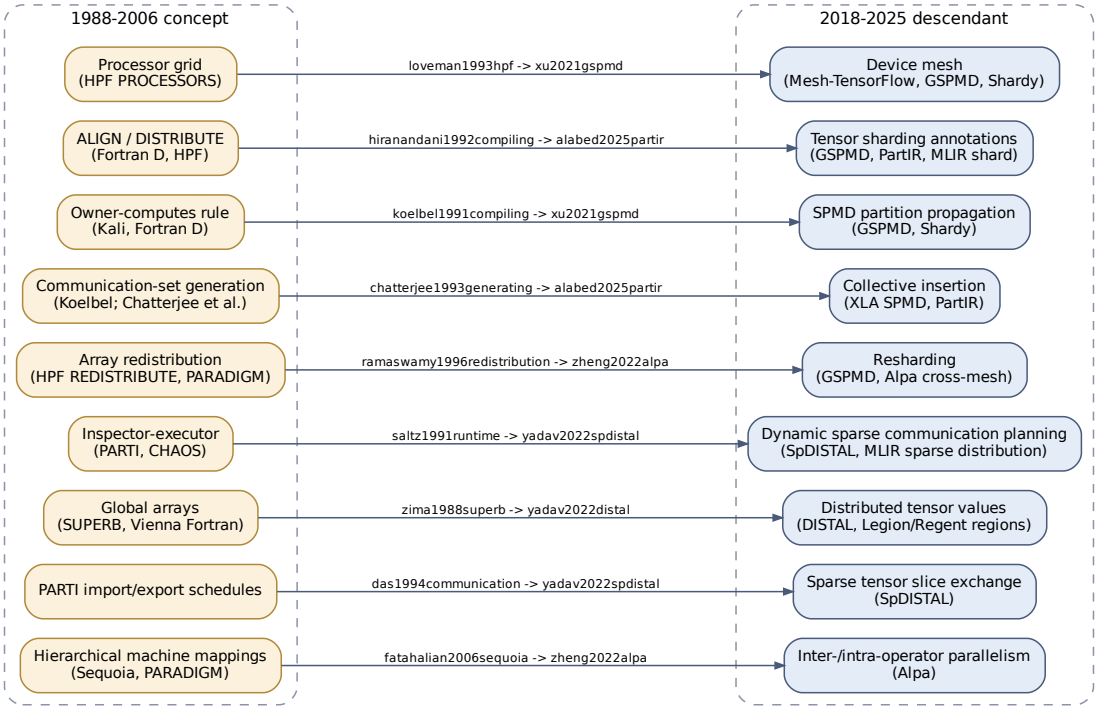


Fig. 5. Concepts of 1988–2006 and their 2018–2025 descendants. Each edge names the catalog entries that document the older and the newer form. The pairing is a survey hypothesis: it asserts functional correspondence, not that the newer work cites the older one; evidence for each pair is tracked in the repository's historical-claims file.

space from local storage persisted from Era 2 through Era 6 (Fig. 5).

B. Ideas that failed or receded

Fully automatic distribution selection for general Fortran programs receded after Era 3; the HPF retrospective attributes HPF's decline to immature compilers, unpredictable performance, and missing support for irregular codes [72]. [TODO: Confirm the attribution against the text of [72] and [11].]

C. Concepts rediscovered under new names

Processor grid → device mesh; ALIGN/DISTRIBUTE → sharding annotation; owner-computes → partition propagation; communication sets → collective insertion; redistribution → resharding; inspector–executor → dynamic sparse communication planning; global arrays → distributed tensor values; PARTI import/export → sparse slice exchange; hierarchical machine mapping → inter-/intra-operator parallelism.

D. Why explicit MPI became dominant, and why high-level partitioning has returned

We hypothesise that MPI won in Era 3–4 because its cost model is transparent to the programmer and its performance does not depend on compiler maturity, whereas high-level partitioning has returned in Era 6 because ML programs are expressed as operator graphs with few, large, regular operators, which makes both propagation and cost modelling tractable in a way that general Fortran loops were not. [TODO: Test this hypothesis against the evaluation sections of [6], [7], [34] and against [72].]

XIII. Open Problems

- **Unified ownership/layout IR.** No representation today spans HPF-style affine distributions, mapping arrays, logical regions, and mesh shardings; MLIR dialects are the current candidates. [TODO: cite MLIR shard and Shardy documentation once verified.]
- **Dynamic and irregular partitioning.** Inspector–executor techniques and sparse tensor distribution formats remain separate lines of work.
- **Hierarchical heterogeneous systems.** Node–device–core hierarchies are handled by nested mappings in Sequoia and Alpa but not by a general mechanism.
- **Topology and memory-capacity modelling.** Cost models in SPMD partitioners seldom model network topology or capacity limits jointly.
- **Correctness and consistency.** Verification of generated communication is rarely addressed.
- **Communication/computation overlap, composability, debuggability, performance portability.**

- **Wafer-scale and fine-grained local-memory machines.** Spatial placement with per-PE memory re-opens the placement problem at a scale where closed-form distributions do not apply.

[TODO: Each item needs evidence from at least one catalog entry stating the limitation.]

XIV. Conclusion

Distributed-memory compilation has been re-invented for each new class of machine with private memories. The catalog, taxonomy, and figures behind this survey are maintained openly so that the next re-invention can begin from the evidence rather than from scratch. [TODO: Finalise once the corpus and synthesis sections are complete.]

References

[1] D. Callahan and K. Kennedy, "Compiling programs for distributed-memory multiprocessors," *The Journal of Supercomputing*, 1988.

[2] H. P. Zima, H.-J. Bast, and M. Gerndt, "Superb: A tool for semi-automatic mimd/simd parallelization," *Parallel Computing*, 1988.

[3] M. Chen, Y.-I. Choo, and J. Li, "Compiling parallel programs by optimizing performance," *The Journal of Supercomputing*, 1988.

[4] D. Loveman, "High performance fortran," *IEEE Parallel & Distributed Technology: Systems & Applications*, 1993.

[5] W. Gropp, E. Lusk, N. Doss, and A. Skjellum, "A high-performance, portable implementation of the mpi message passing interface standard," *Parallel Computing*, 1996.

[6] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, "Gspmd: General and scalable parallelization for ml computation graphs," arXiv preprint, 2021. [Online]. Available: https://arxiv.org/pdf/2105.04663

[7] S. Alabed, D. Belov, B. Chrzaszcz, J. Franco, D. Grewe, D. Maclaurin, J. Molloy, T. Natan, T. Norman, X. Pan, A. Paszke, N. A. Rink, M. Schaarschmidt, T. Sitdikov, A. Swietlik, D. Vytiniotis, and J. Wee, "Partir: Composing spmd partitioning strategies for machine learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025.

[8] R. Yadav, A. Aiken, and F. Kjolstad, "Distal: the distributed tensor algebra compiler," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022.

[9] K. Rocki, D. V. Essendelft, I. Sharapov, R. Schreiber, M. Morrison, V. Kibardin, A. Portnoy, J. F. Dietiker, M. Syamlal, and M. James, "Fast stencil-code computation on a wafer-scale processor," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[10] H. Zima and B. Chapman, "Compiling for distributed-memory systems," *Proceedings of the IEEE*, 1993.

[11] S. Benkner and H. Zima, "Compiling high performance fortran for distributed-memory architectures," *Parallel Computing*, 1999.

[12] D. Unat, A. Dubey, T. Hoefler, J. Shalf, M. Abraham, M. Bianco, B. L. Chamberlain, R. Cledat, H. C. Edwards, H. Finkel, K. Fuerlinger, F. Hannig, E. Jeannot, A. Kamil, J. Keasler, P. H. J. Kelly, V. Leung, H. Ltaief, N. Maruyama, C. J. Newburn, and M. Pericas, "Trends in data locality abstractions for hpc systems," *IEEE Transactions on Parallel and Distributed Systems*, 2017.

[13] P. Lee, "Techniques for compiling programs on distributed memory multicomputers," *Parallel Computing*, 1995.

[14] J. W. Backus, H. Stern, I. Ziller, R. A. Hughes, R. Nutt, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, and P. B. Sheridan, "The fortran automatic coding system," *Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability on - IRE-AIEE-ACM '57 (Western)*, 1957.

[15] L. Lamport, "The parallel execution of do loops," *Communications of the ACM*, 1974.

[16] D. J. Kuck, R. H. Kuhn, D. A. Padua, B. Leasure, and M. Wolfe, "Dependence graphs and compiler optimizations," in *Proceedings of the 8th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '81*, 1981.

[17] R. Allen and K. Kennedy, "Automatic translation of fortran programs to vector form," *ACM Transactions on Programming Languages and Systems*, 1987.

[18] G. Barnes, R. Brown, M. Kato, D. Kuck, D. Slotnick, and R. Stokes, "The illiac iv computer," *IEEE Transactions on Computers*, 1968.

[19] R. M. Russell, "The cray-1 computer system," *Communications of the ACM*, 1978.

[20] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor," *Proceedings of the 2nd annual symposium on Computer architecture - ISCA '75*, 1975.

[21] Kung, "Why systolic architectures?" *Computer*, 1982.

[22] C. L. Seitz, "The cosmic cube," *Communications of the ACM*, 1985.

[23] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, "Legion: Expressing locality and independence with logical regions," in *2012 International Conference for High Performance Computing, Networking, Storage and Analysis*, 2012.

[24] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Compiling fortran d for mimd distributed-memory machines," *Communications of the ACM*, 1992.

[25] U. Bondhugula, "Compiling affine loop nests for distributed-memory parallel architectures," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2013.

[26] G. Bisbas, A. Lydike, E. Bauer, N. Brown, M. Fehr, L. Mitchell, G. Rodriguez-Canal, M. Jamieson, P. H. J. Kelly, M. Steuwer, and T. Grosser, "A shared compilation stack for distributed-memory parallelism in stencil dsls," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024.

[27] C. Koelbel and P. Mehrotra, "Compiling global name-space parallel loops for distributed execution," *IEEE Transactions on Parallel and Distributed Systems*, 1991.

[28] C. Koelbel, "Compile-time generation of regular communications patterns," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[29] S. Chatterjee, J. R. Gilbert, F. J. E. Long, R. Schreiber, and S.-H. Teng, "Generating local addresses and communication sets for data-parallel programs," in *Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming - PPOPP '93*, 1993.

[30] J. Stichnoth, D. Ohallaron, and T. Gross, "Generating communication for array statements: Design, implementation, and evaluation," *Journal of Parallel and Distributed Computing*, 1994.

[31] J. Saltz, K. Crowley, R. Michandaney, and H. Berryman, "Run-time scheduling and execution of loops on message passing machines," *Journal of Parallel and Distributed Computing*, 1990.

[32] S. Ramaswamy, B. Simons, and P. Banerjee, "Optimizations for efficient array redistribution on distributed memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[33] R. Das, M. Uysal, J. Saltz, and Y. Hwang, "Communication optimizations for irregular scientific computations on distributed memory architectures," *Journal of Parallel and Distributed Computing*, 1994.

[34] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin

[35] J. Li and M. Chen, "The data alignment phase in compiling programs for distributed-memory machines," *Journal of Parallel and Distributed Computing*, 1991.

[36] P. Banerjee, J. Chandy, M. Gupta, E. Hodges, J. Holm, A. Lain, D. Palermo, S. Ramaswamy, and E. Su, "The paradigm compiler for distributed-memory multicomputers," *Computer*, 1995.

[37] R. W. Numrich and J. Reid, "Co-array fortran for parallel programming," *ACM SIGPLAN Fortran Forum*, 1998.

[38] K. Yelick, L. Semenzato, G. Pike, C. Miyamoto, B. Liblit, A. Krishnamurthy, P. Hilfinger, S. Graham, D. Gay, P. Colella, and A. Aiken, "Titanium: a high-performance java dialect," *Concurrency: Practice and Experience*, 1998.

[39] P. Tang and J. Xue, "Generating efficient tiled code for distributed memory machines," *Parallel Computing*, 2000.

[40] W.-Y. Chen, D. Bonachea, J. Duell, P. Husbands, C. Iancu, and K. Yelick, "A performance analysis of the berkeley upc compiler," in *Proceedings of the 17th annual international conference on Supercomputing*, 2003.

[41] M. Kruse, "Introducing molly: Distributed memory parallelization with llvm," arXiv preprint, 2014. [Online]. Available: https://arxiv.org/pdf/1409.2088

[42] U. Bondhugula, "Automatic distributed-memory parallelization and code generation using the polyhedral framework," IISc-CSA-TR-2011-3, Indian Institute of Science, Tech. Rep., 2011. [Online]. Available: https://www.csa.iisc.ac.in/~udayb/publications/uday11distmem-tr.pdf

[43] M. Gerndt, "Updating distributed variables in local computations," *Concurrency: Practice and Experience*, 1990.

[44] P. Mehrotra and J. V. Rosendale, "Programming distributed memory architectures using kali," NASA Contractor Report 187453 / ICASE Report 90-69, Tech. Rep., 1990. [Online]. Available: https://ntrs.nasa.gov/citations/19910002904

[45] J. Saltz, H. Berryman, and J. Wu, "Multiprocessors and run-time compilation," *Concurrency: Practice and Experience*, 1991.

[46] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Compiler optimizations for fortran d on mimd distributed-memory machines," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[47] B. Chapman, P. Mehrotra, and H. Zima, "Vienna fortran: A fortran language extension for distributed memory multiprocessors," NASA Contractor Report 187634 / ICASE Report 91-72, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002452

[48] ——, "Programming in vienna fortran," *Scientific Programming*, 1992.

[49] M. Gupta and P. Banerjee, "Demonstration of automatic data partitioning techniques for parallelizing compilers on multicomputers," *IEEE Transactions on Parallel and Distributed Systems*, 1992.

[50] E. Su, D. J. Palerrno, and P. Banerjee, "Automating parallelism of regular computations for distributed-memory multicomputers in the paradigm compiler," in *1993 International Conference on Parallel Processing - ICPP'93 Vol2*, 1993.

[51] Z. Bozkus, A. Choudhary, G. Fox, T. Haupt, and S. Ranka, "Fortran 90d/hpf compiler for distributed memory mimd computers," in *Proceedings of the 1993 ACM/IEEE conference on Supercomputing - Supercomputing '93*, 1993.

[52] H. P. F. Forum, "High performance fortran language specification," *ACM SIGPLAN Fortran Forum*, 1993.

[53] J. Harris, J. A. Bircsak, M. R. Bolduc, J. A. Diewald, I. Gale, N. W. Johnson, S. Lee, C. A. Nelson, and C. D. Offner, "Compiling high performance fortran for distributed-memory systems," *Digital Technical Journal*, 1995. [Online]. Available: https://ftp.math.utah.edu/pub/tex/bib/idx/dectechj/7/3/5_23.html

[54] P. Charles, C. Grothoff, V. Saraswat, C. Donawa, A. Kielstra, K. Ebcioglu, C. von Praun, and V. Sarkar, "X10: an object-oriented approach to non-uniform cluster computing," *Proceedings of the 20th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, 2005.

[55] B. Chamberlain, D. Callahan, and H. Zima, "Parallel programmability and the chapel language," *The International Journal of High Performance Computing Applications*, 2007.

[56] K. Fatahalian, T. Knight, M. Houston, M. Erez, D. Horn, L. Leem, J. Park, M. Ren, A. Aiken, W. Dally, and P. Hanrahan, "Sequoia: Programming the memory hierarchy," in *ACM/IEEE SC 2006 Conference (SC'06)*, 2006.

[57] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan, "A practical automatic polyhedral parallelizer and locality optimizer," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2008.

[58] E. Slaughter, W. Lee, S. Treichler, M. Bauer, and A. Aiken, "Regent: a high-productivity programming language for hpc with logical regions," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015.

[59] N. Shazeer, Y. Cheng, N. Parmar, D. Tran, A. Vaswani, P. Koanantakool, P. Hawkins, H. Lee, M. Hong, C. Young, R. Sepassi, and B. Hechtman, "Mesh-tensorflow: Deep learning for supercomputers," arXiv preprint (also NeurIPS 2018), 2018. [Online]. Available: https://arxiv.org/pdf/1811.02084

[60] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," Proceedings of Machine Learning and Systems (MLSys) 1, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/b422680f3db0986ddd7f8f126baaf0fa-Abstract.html

[61] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," arXiv preprint (also ICLR 2021), 2020. [Online]. Available: https://arxiv.org/pdf/2006.16668

[62] M. Jacquelin, M. Araya–Polo, and J. Meng, "Scalable distributed high-order stencil computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[63] R. Yadav, A. Aiken, and F. Kjolstad, "Spdistal: Compiling distributed sparse tensor computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[64] L. community, "Mlir 'shard' dialect (official documentation; formerly 'mesh')," MLIR documentation (mlir.llvm.org), 2025. [Online]. Available: https://mlir.llvm.org/docs/Dialects/Shard/

[65] N. Stawinoga, D. Katz, A. Lydike, J. Zarins, N. Brown, G. Bisbas, and T. Grosser, "An mlir lowering pipeline for stencils at wafer-scale," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, 2026.

[66] O. Project, "Shardy: an mlir-based tensor partitioning system (official documentation)," OpenXLA project documentation, 2024. [Online]. Available: https://openxla.org/shardy

[67] W. D. Hillis and G. L. Steele, "Data parallel algorithms," *Communications of the ACM*, 1986.

[68] J. Ramanujam and P. Sadayappan, "Compile-time techniques for data distribution in distributed memory machines," *IEEE Transactions on Parallel and Distributed Systems*, 1991.

[69] M. Gupta, S. Midkiff, E. Schonberg, V. Seshadri, D. Shields, K.-Y. Wang, W.-M. Ching, and T. Ngo, "An hpf compiler for the ibm sp2," in *Proceedings of the 1995 ACM/IEEE conference on Supercomputing (CDROM) - Supercomputing '95*, 1995.

[70] D. J. Palermo, E. W. H. IV, and P. Banerjee, "Dynamic data partitioning for distributed-memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[71] R. Ponnusamy, J. Saltz, A. Choudhary, Y.-S. Hwang, and G. Fox, "Runtime support and compilation methods for user-specified irregular data distributions," *IEEE Transactions on Parallel and Distributed Systems*, 1995.

[72] K. Kennedy, C. Koelbel, and H. Zima, "The rise and fall of high performance fortran," in *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, 2007.

[73] M. M. Baskaran, U. Bondhugula, S. Krishnamoorthy, J. Ramanujam, A. Rountev, and P. Sadayappan, "Automatic data movement and computation mapping for multi-level parallel architectures with explicitly managed memories," in *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming*, 2008.

[74] R. Dathathri, R. T. Mullapudi, and U. Bondhugula, "Compiling affine loop nests for a dynamic scheduling runtime on shared and distributed memory," *ACM Transactions on Parallel Computing*, 2016.

[75] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," arXiv preprint (also NeurIPS 2019), 2018. [Online]. Available: https://arxiv.org/pdf/1811.06965

[76] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "Pipedream: generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

[77] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," arXiv preprint, 2019. [Online]. Available: https://arxiv.org/pdf/1909.08053

[78] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[79] M. Kong, R. A. Yosef, A. Rountev, and P. Sadayappan, "Automatic generation of distributed-memory mappings for tensor computations," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023.

TABLE II
CORE SYSTEMS IN THE CATALOG. A QUESTION MARK MARKS A CELL NOT YET VERIFIED FROM A PRIMARY SOURCE.

System	Refs	Years	Input	Distribution	Work mapping	Communication	Runtime	Targets	Artifa
SUPERB	[2], [43]	1986-1992	Fortran 77 + interactive distribution	?	?	?	SUPRENUM MP	SUPRENUM	unavai
Kali	[27], [44]	1989-1992	Fortran + dist + doall/on	block, cyclic, block_cyclic, user map, dynamic	on clause / owner()	in/out sets static or run-time inspector	cached inspector/executor	iPSC-class multicomputers	unavai
PARTI / CHAOS	[31], [45]	1989-1997	irregular loops with index arrays	mapping arrays via translation tables; partitioner-driven	iteration partitioning via RIG/partitioner	inspector-built import/export schedules; schedule reuse	PARTI/CHAOS	iPSC/2, iPSC/860, CM-2	unavai
Fortran D	[1], [46]	1991-1996	Fortran 77 + decomposition directives	?	?	?	NX / PARTI	iPSC/860	unavai
Vienna Fortran / VFCS	[47], [48]	1991-1999	Fortran 77 + DIST/ALIGN	block, cyclic(k), user-defined, INDIRECT, dynamic	owner computes; FORALL ON	overlap areas, vectorised messages (VFCS)	VFCS	iPSC/860, GENESIS, SUPRENUM	unavai
PARADIGM	[49], [50]	1992-1997	Fortran 77 (automatic)	?	?	?	?	iPSC, Paragon, CM-5 (to verify)	unavai
Fortran 90D/HPF (Syracuse)	[51]	1993-1995	Fortran 90D	?	?	runtime collective library	F90D runtime	?	unavai
High Performance Fortran (language)	[4], [52]	1993-2000	Fortran 90 + directives	BLOCK, CYCLIC, CYCLIC(k), replicated; 2.0 adds INDIRECT, GEN_BLOCK	implementation-defined (owner computes typical)	implementation-defined	n/a	n/a	specifi
Digital (DEC) Fortran 90 HPF compiler	[53]	1994-1998	HPF 1.1	BLOCK, CYCLIC, replicated, alignment	data-parallel; owner computes not strictly enforced	region assignments; vectorised sends; shadow edges; inlined reductions	PSE SEND/RECEIVE	Alpha workstation farms, AlphaServer SMP	comm unavai
Titanium	[38]	1998-2010	Titanium (Java dialect)	?	SPMD	?	GASNet	clusters	archiv
Co-array Fortran	[37]	1998-present	Fortran + [ ] codimensions	one co-array per image	SPMD, programmer	explicit co-array references	?	Cray T3E and later	spec; 2008
Unified Parallel C (Berkeley UPC)	[40]	2002-present	UPC	?	upc_forall affinity	?	GASNet	clusters	open s
X10	[54]	2004-2020	X10	?	at(place) / async	?	X10RT	clusters	archiv
Chapel	[55]	2005-present	Chapel	user-defined domain maps (Block, Cyclic, BlockCyclic, Replicated, ...)	forall over distributed domains; on clauses	compiler + runtime (GET/PUT, aggregation)	Chapel runtime	clusters, GPUs	open s
Sequoia	[56]	2006-2010	task trees	?	task-to-memory-level mapping	bulk copies between levels	Sequoia runtime	Cell, clusters	unavai
Pluto distributed-memory code generation (Bondhugula et al.)	[42], [57]	2011-2016	C affine loop nests	derived from computation partition (flow-out sets)	polyhedral iteration-space partitioning	polyhedral communication sets to MPI	MPI / dynamic scheduler	InfiniBand clusters	open (pluto distme branch
Legion / Regent	[23], [58]	2012-present	tasks + logical regions	region partitions chosen by program/mapper	mapper interface	runtime-managed coherence and copies	Legion/Realm	GPU clusters	open s
Molly	[41]	2014-2015	C++ + layout declarations	programmer-declared	polyhedral	generated asynchronous MPI	MPI	Blue Gene/Q (intended)	archiv
Mesh-TensorFlow	[59]	2018-2021	TF graph + named dims + layout	dimension-to-mesh-axis layout	SPMD per layout	allreduce/allgather/alltoall derived from layouts	TF/TPU	TPUv2 pods	archiv
FlexFlow	[60]	2018-present	operator graph	SOAP output-tensor partitions	MCMC search with simulator	task graph with transfer tasks	FlexFlow/Legion	P100/K80 clusters	open s
XLA SPMD partitioner (GShard / GSPMD)	[6], [61]	2020-present	HLO + sharding annotations	replicated / tiled / partially tiled per tensor over device mesh	SPMD; sharding propagation	collectives + halo exchange + resharding inserted per operator	XLA	TPUv3 pods (GPU supported)	open (XLA)
Cerebras WSE programming (CSL / hand-mapped stencils)	[9], [62]	2020-present	CSL	programmer-placed per-PE tiles	spatial placement	fabric colours / routes configured by programmer	SDK	CS-1/CS-2/CS-3	comm SDK
PartIR	[7]	2021-2025	StableHLO + tactics	?	?	?	XLA	TPU, GPU	?
DISTAL / SpDISTAL	[8], [63]	2022-2023	tensor index notation + formats + schedule	tensor distribution notation over a processor grid; sparse non-zero/universe partitions	distribute/communicate/rotate schedule to index-task launches	Legion partitions (bounds analysis; image/preimage for sparse)	Legion	Lassen CPU/GPU nodes	open s
Alpa	[34]	2022-2023	JAX graph	per-stage sharding specs (ILP)	DP over stages/submeshes + ILP intra-op	GSPMD collectives + cross-mesh resharding	Alpa MPMD runtime	64x V100	archiv
MLIR shard dialect (formerly mesh)	[64]	2023-present	MLIR + shard.grid/shard.sharding	per-axis grid sharding with halos	SPMD	all_gather, all_reduce, all_to_all, reduce_scatter, update_halo, send/recv	n/a	n/a	upstre MLIR
xDSL stencil / dmp / mpi stack	[26], [65]	2023-present	stencil DSLs	static Cartesian rank-grid decomposition (dmp)	owner computes per block; one z-column per PE on WSE	dmp.swap halo exchange to mpi dialect or CSL chunked prefetch	MPI / CSL library	ARCHER2, GPUs, FPGA, CS-2/CS-3	open s

TABLE III
DECOMPOSITION SUPPORT BY SYSTEM. ● SUPPORTED, ◑ PARTIAL, ○ NOT SUPPORTED, ? NOT YET VERIFIED.

System	block	cyclic	block-cyclic	replication	halo
SUPERB	?	?	?	?	?
Kali	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
PARTI / CHAOS	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$circ$
Fortran D	?	?	?	?	?
Vienna Fortran / VFCS	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
PARADIGM	?	?	?	?	?
Fortran 90D/HPF (Syracuse)	?	?	?	?	?
High Performance Fortran (language)	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
Digital (DEC) Fortran 90 HPF compiler	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$
Titanium	?	?	?	?	?
Co-array Fortran	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
Unified Parallel C (Berkeley UPC)	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$circ$	\{}$circ$
X10	?	?	?	?	?
Chapel	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$bullet$	\{}$circ$\{}!\{}!\{}$bull$
Sequoia	?	?	?	?	?
Pluto distributed-memory code generation (Bondhugula et al.)	\{}$bullet$	?	?	\{}$bullet$	\{}$bullet$
Legion / Regent	\{}$bullet$	\{}$circ$\{}!\{}!\{}$bullet$	\{}$circ$\{}!\{}!\{}$bullet$	\{}$bullet$	\{}$circ$
Molly	\{}$bullet$	?	?	?	?
Mesh-TensorFlow	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
FlexFlow	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
XLA SPMD partitioner (GShard / GSPMD)	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
Cerebras WSE programming (CSL / hand-mapped stencils)	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$bullet$
PartIR	?	?	?	?	?
DISTAL / SpDISTAL	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
Alpa	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
MLIR shard dialect (formerly mesh)	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$bullet$	\{}$circ$
xDSL stencil / dmp / mpi stack	\{}$bullet$	\{}$circ$	\{}$circ$	\{}$circ$	\{}$circ$
Shardy (OpenXLA)	\{}$bullet$	?	?	\{}$bullet$	?

TABLE IV
COMPILER ARCHITECTURE BY SYSTEM (FROM CATALOG TAGS).

System	src-to-src	polyhedral	grap
SUPERB	\{}$bullet$	\{}$circ$	\{}
Kali	\{}$bullet$	\{}$circ$	\{}
PARTI / CHAOS	\{}$circ$	\{}$circ$	\{}
Fortran D	\{}$bullet$	\{}$circ$	\{}
Vienna Fortran / VFCS	\{}$bullet$	\{}$circ$	\{}
PARADIGM	\{}$bullet$	\{}$circ$	\{}
Fortran 90D/HPF (Syracuse)	\{}$bullet$	\{}$circ$	\{}
High Performance Fortran (language)	\{}$circ$	\{}$circ$	\{}
Digital (DEC) Fortran 90 HPF compiler	\{}$circ$	\{}$circ$	\{}
Titanium	\{}$bullet$	\{}$circ$	\{}
Co-array Fortran	\{}$circ$	\{}$circ$	\{}
Unified Parallel C (Berkeley UPC)	\{}$bullet$	\{}$circ$	\{}
X10	\{}$bullet$	\{}$circ$	\{}
Chapel	\{}$bullet$	\{}$circ$	\{}
Sequoia	\{}$circ$	\{}$circ$	\{}
Pluto distributed-memory code generation (Bondhugula et al.)	\{}$bullet$	\{}$bullet$	\{}
Legion / Regent	\{}$circ$	\{}$circ$	\{}
Molly	\{}$circ$	\{}$bullet$	\{}
Mesh-TensorFlow	\{}$circ$	\{}$circ$	\{}b
FlexFlow	\{}$circ$	\{}$circ$	\{}b
XLA SPMD partitioner (GShard / GSPMD)	\{}$circ$	\{}$circ$	\{}b
Cerebras WSE programming (CSL / hand-mapped stencils)	\{}$circ$	\{}$circ$	\{}
PartIR	\{}$circ$	\{}$circ$	\{}
DISTAL / SpDISTAL	\{}$circ$	\{}$circ$	\{}
Alpa	\{}$circ$	\{}$circ$	\{}b
MLIR shard dialect (formerly mesh)	\{}$circ$	\{}$circ$	\{}
xDSL stencil / dmp / mpi stack	\{}$circ$	\{}$circ$	\{}
Shardy (OpenXLA)	\{}$circ$	\{}$circ$	\{}

TABLE V
EVALUATION METHODOLOGY BY SYSTEM. A QUESTION MARK MARKS A CELL NOT YET EXTRACTED FROM THE PRIMARY SOURCE.

System	Benchmarks	Baseline	Scale	Metrics	Productivity eval.	Artifact
SUPERB	?	?	?	?	?	unavailable
Kali	?	?	?	?	?	unavailable
PARTI / CHAOS	?	?	?	?	?	unavailable
Fortran D	?	?	?	?	?	unavailable
Vienna Fortran / VFCS	?	?	?	?	?	unavailable
PARADIGM	?	?	?	?	?	unavailable
Fortran 90D/HPF (Syracuse)	?	?	?	?	?	unavailable
High Performance Fortran (language)	?	?	?	?	?	specification
Digital (DEC) Fortran 90 HPF compiler	?	?	?	?	?	commercial, unava
Titanium	?	?	?	?	?	archived
Co-array Fortran	?	?	?	?	?	spec; Fortran 2008 rays
Unified Parallel C (Berkeley UPC)	?	?	?	?	?	open source
X10	?	?	?	?	?	archived
Chapel	?	?	?	?	?	open source
Sequoia	?	?	?	?	?	unavailable
Pluto distributed-memory code generation (Bondhugula et al.)	?	?	?	?	?	open source (pluto mem branch)
Legion / Regent	?	?	?	?	?	open source
Molly	?	?	?	?	?	archived
Mesh-TensorFlow	?	?	?	?	?	archived
FlexFlow	?	?	?	?	?	open source
XLA SPMD partitioner (GShard / GSPMD)	?	?	?	?	?	open source (XLA
Cerebras WSE programming (CSL / hand-mapped stencils)	?	?	?	?	?	commercial SDK
PartIR	?	?	?	?	?	?
DISTAL / SpDISTAL	?	?	?	?	?	open source
Alpa	?	?	?	?	?	archived
MLIR shard dialect (formerly mesh)	?	?	?	?	?	upstream MLIR
xDSL stencil / dmp / mpi stack	?	?	?	?	?	open source
Shardy (OpenXLA)	?	?	?	?	?	open source